INCIDENT LIFE CYCLE AUTOMATION

Final Project Report

Consolidated report for Incident Life Cycle Automation

STUDENT NAME **Shanmugapriya M**	PROJECT **Incident Life Cycle Automation**
DOCUMENT DATE **08 October 2026**	SCOPE **Incident lifecycle workflow & automation only**

1. Introduction

1.1 Project Overview

Incident Life Cycle Automation is a workflow-oriented system for managing operational incidents as one continuous, traceable process. It focuses on reducing manual coordination by automating intake, triage, assignment, SLA monitoring, escalation, resolution and closure.

1.2 Purpose

The purpose is to improve response visibility, ownership, consistency and auditability without adding modules unrelated to incident lifecycle management.

2. Ideation Phase

2.1 Problem

Incident handling can be fragmented across manual steps, causing delayed ownership, unclear status, late escalation and incomplete history.

2.2 Empathy Insight

Reporters need a simple entry point and visibility; support engineers need context and ownership; incident managers need early SLA risk visibility; reviewers need reliable history.

2.3 Selected Ideas

The scope prioritizes structured intake, automated routing, lifecycle status, SLA monitoring, escalation, resolution, closure and audit history.

3. Requirement Analysis

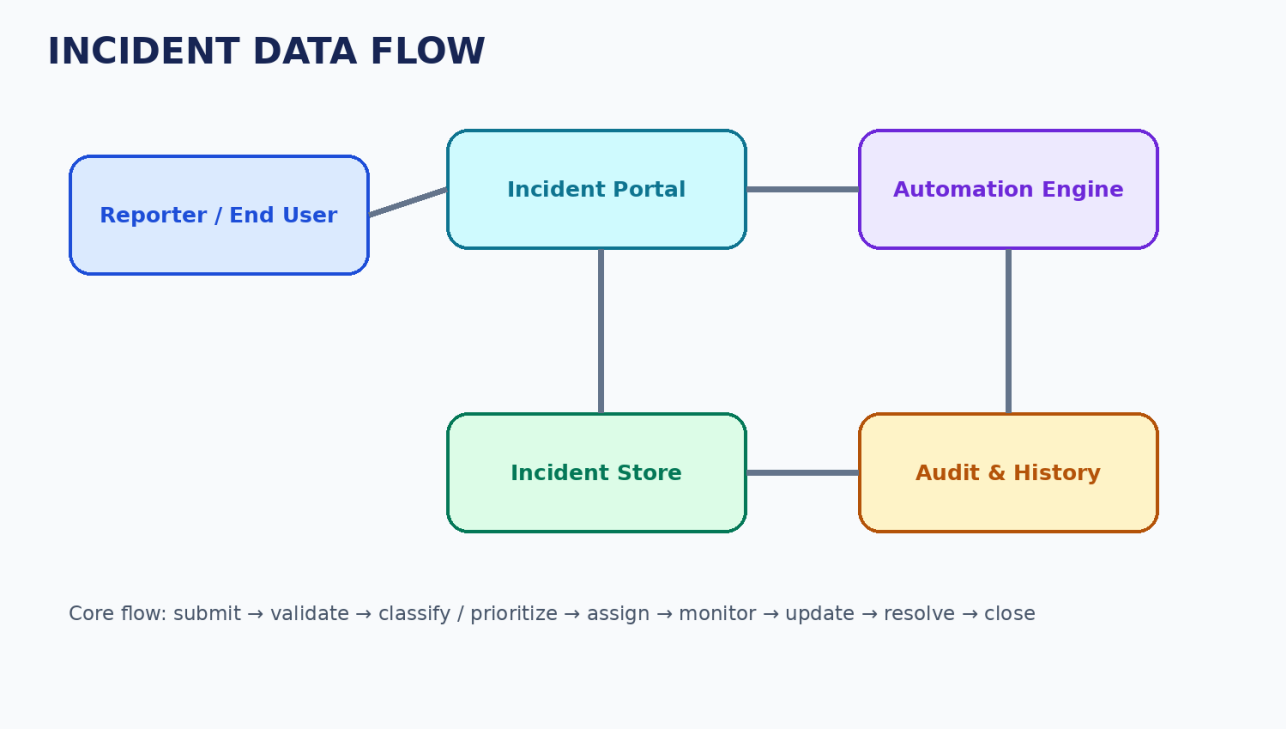
3.1 Journey

Notice issue → report → receive ownership → track progress → respond to SLA risk → resolve → close and review.

3.2 Requirements

Core requirements cover incident creation, identification, priority, assignment, SLA tracking, escalation, investigation, resolution, closure, history, search/dashboard, role access and auditability.

3.3 Data Flow



3.4 Technology

HTML/CSS/JavaScript for the interface, Python/Flask for application logic, SQLite for prototype persistence, a background scheduler for SLA checks and email delivery for notifications.

4. Project Design

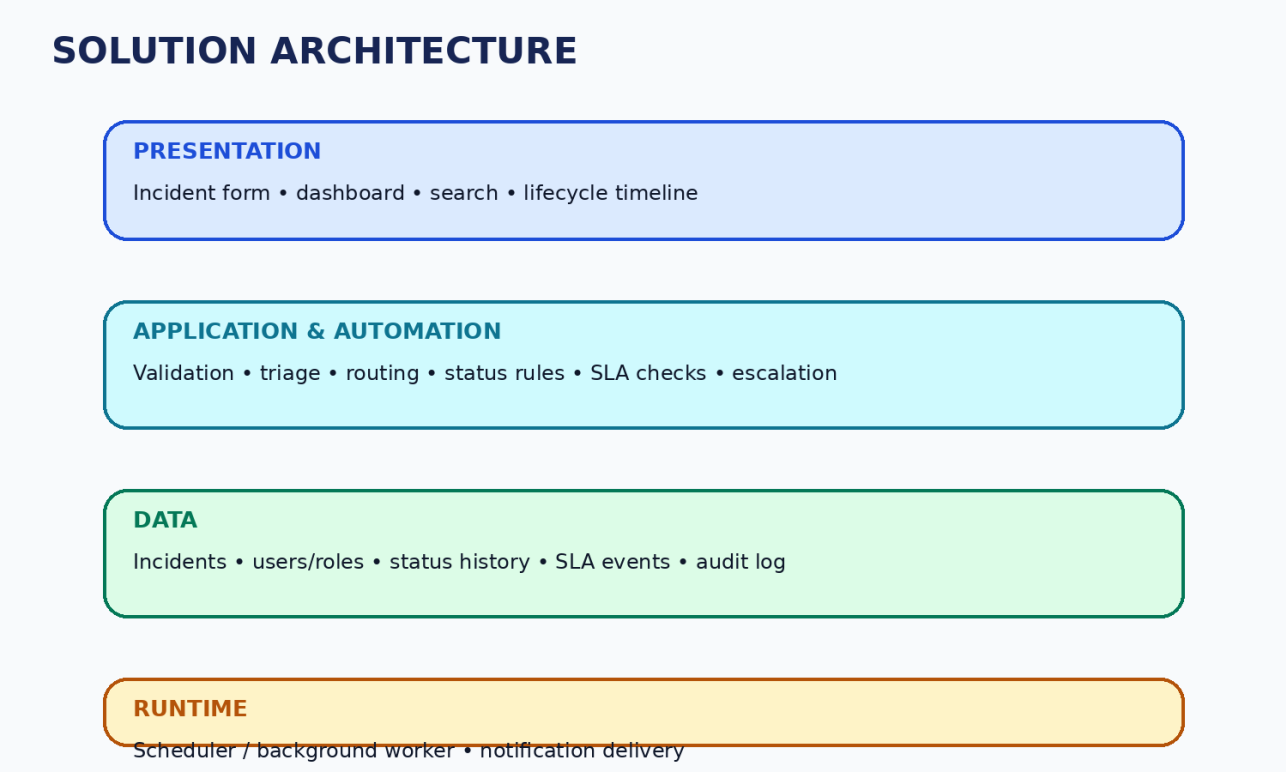
4.1 Problem–Solution Fit

Structured intake reduces missing information; routing reduces ownership delay; SLA automation reduces late follow-up; lifecycle history improves traceability.

4.2 Proposed Solution

The system captures incidents, applies lifecycle rules, assigns ownership, monitors SLA, records investigation and resolution, and closes each incident while retaining its history.

4.3 Architecture



5. Project Planning

Sprint	Focus	Points
Sprint 1	Intake, validation, priority and assignment	16
Sprint 2	Status, investigation, SLA, resolution and closure	16
Sprint 3	Dashboard, audit, rules and notifications	12

6. Testing

Functional tests cover positive and negative incident flows. Performance testing measures common actions and background SLA processing. UAT validates the end-to-end user workflow. Actual results should be attached after execution.

7. Expected Results

- Incident records move through a defined lifecycle.
- Ownership and status remain visible.
- SLA risk can trigger escalation.
- Resolution and closure preserve history.
- Authorized users can review lifecycle activity.

8. Advantages & Limitations

Advantages	Limitations
Clear ownership and lifecycle visibility	Baseline scope is limited to incident automation
Less repetitive manual follow-up	Advanced integrations are outside the core build
SLA-driven escalation	Performance depends on the actual environment
Consistent history	Rules require correct configuration

9. Conclusion

Incident Life Cycle Automation turns a fragmented incident-handling process into a focused, rule-driven workflow. The project is designed to improve visibility, reduce manual coordination and preserve a dependable history from creation to closure.

10. Future Scope

Possible next steps include service-specific SLA policies, richer trend reporting and carefully selected external integrations, after the core lifecycle is validated.

11. Appendix

Attach the actual source code, implementation screenshots and executed test evidence. Add a demo link only when a real deployment exists.